

Алгоритмы и структуры данных—1

Коллоквиум

Материалы для подготовки

Винер Даниил [@danya_vin](#)

Версия от 1 июня 2026 г.

Материал в этом файле задает базовый минимум знаний, необходимых для коллоквиума. Принимающий может проверять не только знание изложенных тем, но и глубину их понимания с помощью уточняющих вопросов.

Содержание

1	Способы задания графов	3
1.1	Список ребер	3
1.2	Матрица смежности	3
1.3	Список смежности	3
2	Обход в глубину	4
2.1	Обход в глубину	4
2.2	Связность неориентированного графа	5
2.3	Сильная связность ориентированного графа	5
2.4	Поиск компонент связности в графе	6
2.5	Поиск цикла в графе	6
2.6	Проверка графа на двудольность	7
2.7	Диаметр и центр дерева	7
3	Задача построения дерева кратчайших расстояний	8
3.1	Обход в ширину	8
3.2	Алгоритм Дейкстры	9
3.3	Алгоритм Форда–Беллмана	10
3.4	Алгоритм Флойда–Уоршалла	11
4	Задача union — find	12
4.1	Задача union — find	12
4.2	Наивная реализация	12
4.3	Реализация с использованием линейных списков	12
4.4	Система непересекающихся множеств	13
4.4.1	Сжатие путей	13
4.4.2	Объединение деревьев	14
4.5	Алгоритм Краскала	14
5	Дерево отрезков	15
5.1	Дерево отрезков	15
5.2	Операции на отрезке	15
5.2.1	Построение	15
5.2.2	Изменение	16
5.2.3	Сумма	16
5.3	Применение дерева отрезков	16
6	Дерево поиска	17
6.1	Поиск элемента	17

6.2	Вставка элемента	17
6.3	Удаление элемента	18
7	Декартово дерево	19
7.1	По явному ключу	19
7.2	По неявному ключу	19
7.3	Операции	20
	7.3.1 Merge	20
	7.3.2 Split	20
7.4	Применение декартового дерева	21
	7.4.1 Вставка	21
	7.4.2 Сумма на отрезке	21
	7.4.3 Переворот	21

1 Способы задания графов

Любой граф G можно представить двумя множествами: V — множество вершин, E — множество рёбер. Будем обозначать $n = |V|$ — количество вершин, $m = |E|$ — количество рёбер.

1.1 Список ребер

Определение. Список ребер — структура данных, представляющая собой набор из пар $A_i - B_i$, где A_i, B_i — вершины графа

Порядок в списке ребер не важен **только в случае неориентированных** графов

Сложность нахождения всех соседей каждой вершины

Если граф неориентирован, то оптимально будет сделать его ориентированным (помимо ребра $A_i - B_i$ добавить ребро $B_i - A_i$)

- Для поиска всех соседей вершины (в неориентированном случае) надо перебрать все ребра и сравнить текущую вершину со второй вершиной ребра. $O(m)$
- Сортировка ребер — $O(m \log m)$
- Поиск первого соседа в списке ребер для одной вершины — $O(\log m)$
- Поиск первого соседа в списке ребер для всех — $O(n \log m)$

Так как обычно в графах $m \geq n$, то для поиска всех соседей всех вершин потребуется $O(m \log m)$

1.2 Матрица смежности

Матрица смежности — матрица графа $G = (V, E)$ размера $n \times n$, такая что на пересечении i -й строки и j -го столбца стоит 1, если есть ребро между вершинами i и j , и 0 иначе

Если граф ориентированный, то $A[i][j] = 1$ означает наличие ребра $i \rightarrow j$. Для неориентированного графа матрица смежности симметрична относительно главной диагонали.

Имея граф, заданный матрицей смежности, можно выполнить такие операции:

- Проверить наличие ребра между двумя вершинами за $O(1)$
- Найти всех соседей одной вершины за $O(n)$
- Просмотреть все рёбра графа за $O(n^2)$

Затраты по памяти — $O(n^2)$, так как храним матрицу размера $n \times n$

1.3 Список смежности

Список смежности — массив, в котором каждой вершине графа соответствует список, состоящий из соседей этой вершины

Изначально создается n динамически расширяемых пустых векторов. При считывании ребра $A_i - B_i$ в вектор с номером A_i добавляется ребро B_i (если граф неориентированный, то еще в вектор с номером B_i добавляется ребро A_i)

Сложность построения — $O(n + m)$, так как нужно создать n списков и добавить m рёбер. Если пустые списки уже созданы, добавление всех рёбер занимает $O(m)$

Сложность нахождения всех соседей вершины — $O(\deg v)$, где $\deg v$ — степень вершины v

Сложность нахождения всех соседей каждой вершины — $O(n + m)$, так как мы проходим по всем вершинам за n и просматриваем каждое ребро за m

Затраты по памяти — $O(n + m)$

2 Обход в глубину

2.1 Обход в глубину

Описание алгоритма

Обход начинается с выбранной стартовой вершины. Из текущей вершины алгоритм переходит в одного из непосещенных соседей. Если у вершины не осталось непосещенных соседей, то алгоритм возвращается назад по текущему пути, пока не найдет вершину, из которой можно продолжить обход.

Рекурсивная реализация. При входе в вершину v помечаем ее посещенной. Затем перебираем всех соседей to вершины v ; если сосед еще не посещен, рекурсивно запускаем DFS из to . Когда у вершины не остается непосещенных соседей, рекурсивный вызов завершается, и управление возвращается к предыдущей вершине в текущем пути.

Итеративная реализация на стеке. В стек кладем стартовую вершину и помечаем ее посещенной. Пока стек не пуст, смотрим на вершину v наверху стека. Если у v есть непосещенный сосед to , помечаем to посещенным и кладем его в стек. Если таких соседей нет, удаляем v из стека и возвращаемся к предыдущей вершине. Так стек явно хранит текущий путь обхода и заменяет стек рекурсивных вызовов. Чтобы не перебирать одних и тех же соседей заново, для каждой вершины хранят номер следующего рассматриваемого соседа или кладут в стек пару из вершины и этого номера.

Отметим, что в общем случае, когда неизвестно, связный граф или нет, нужно запустить обход в цикле по всем вершинам: новый запуск начинается из каждой еще непосещенной вершины

Применение алгоритма

1. Поиск случайного пути в лабиринте
2. Решение задач, связанных с построением маршрута: в сети, на карте, в сервисах покупки билетов и так далее
3. Проверка на наличие циклов, топологическая сортировка

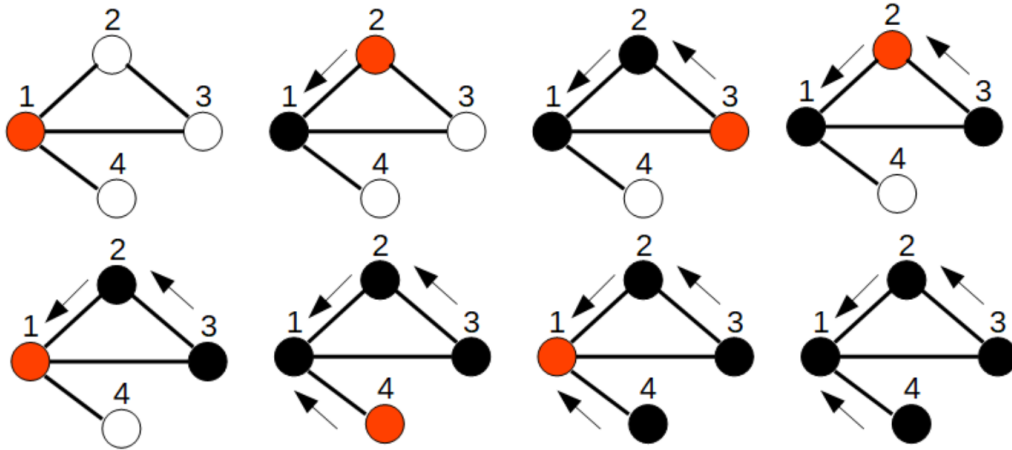
Асимптотика

Дополнительная память DFS — $O(V)$, где V — количество вершин графа. Эта оценка учитывает массив посещенности и стек рекурсии или явный стек, но не учитывает память на хранение самого графа.

Временная сложность зависит от представления графа: матрица смежности, список ребер или список смежности. Рассмотрим каждый вариант:

1. Список смежности — $O(V + E)$
 - Вершины посещаются (проверяются на посещенность) только один раз, что составляет $O(V)$
 - Суммарно просматриваются все записи списков смежности: в ориентированном графе это E записей, а в неориентированном — $2E$, что все равно составляет $O(E)$
2. Матрица смежности — $O(V^2)$
 - Для каждой посещенной вершины приходится просматривать строку матрицы длины V ; при обходе всех компонент таких строк не больше V
3. Список рёбер
 - Без предварительной обработки для поиска соседей текущей вершины приходится просматривать весь список рёбер, поэтому полный обход может занять $O(VE)$
 - Обычно перед DFS по списку рёбер строят список смежности за $O(V + E)$, после чего сам обход работает за $O(V + E)$
 - Если хранить отсортированный список рёбер и искать диапазон исходящих рёбер бинарным поиском, то с учетом сортировки получается $O(E \log E + V \log E + E)$

Пример



2.2 Связность неориентированного графа

Связный граф — граф, в котором существует путь от любой вершины до любой другой вершины

Проверка неориентированного графа на связность

Запуск DFS от любой из вершин графа

- Запускаем DFS из любой вершины
- После DFS проверяем, посещены ли все вершины. Если да — граф связный, нет — несвязный
- Дополнительная память — $O(V)$ под массив посещенности и стек рекурсии или явный стек, плюс хранение графа
- Сложность по времени при хранении графа списком смежности — $O(V + E)$

2.3 Сильная связность ориентированного графа

Определение. В ориентированном графе сильная связность требует, чтобы для любых двух вершин u и v существовал ориентированный путь $u \rightarrow v$ и $v \rightarrow u$

Для проверки нужно выполнить два запуска DFS

1. Запускаем DFS на исходном графе
 - Выбираем любую вершину v
 - Запускаем из неё DFS, пометая все достижимые вершины
 - После завершения проверяем: если посетили не все V вершин, то граф не сильно связан
2. Второй запуск DFS в транспонированном графе
 - Строим транспонированный граф, то есть все ребра $u \rightarrow v$ превращаются в $v \rightarrow u$
 - Запускаем DFS из той же вершины v
 - Снова убеждаемся, что посетили все вершины

Если во время обоих обходов мы посетили все вершины, то граф является сильно связным

Временная сложность: $O(V + E)$ при хранении графа списком смежности, так как построение транспонированного графа и каждый из двух запусков DFS занимают линейное время.

Дополнительная память: $O(V + E)$, если транспонированный граф строится явно. Без учета транспонированного графа сами обходы требуют $O(V)$ памяти.

2.4 Поиск компонент связности в графе

Компонента связности неориентированного графа — максимальное по включению подмножество вершин и соединяющих их ребер, такое что есть путь из каждой вершины в каждую

Алгоритм поиска компоненты связности

Можно раскрасить граф в компоненты связности. Каждой вершине ставится в соответствие номер компоненты связности, к которой относится вершина

Реализация происходит через DFS

Может потребоваться несколько запусков поиска в глубину. Поэтому поиск проводим через цикл, перебирающий все вершины

- Заводим массив длины V для хранения цвета каждой вершины
- При входе в DFS красим вершину в текущий цвет (`color[v] = component`)
- Если очередная вершина не покрашена, то счетчик количества компонент связности увеличивается и запускается DFS для этой вершины со значением цвета равным текущему значению счетчика

Затраты по памяти составят $O(V)$ под массив цветов и стек рекурсии (или стек при итеративном обходе), плюс хранение графа

Сложность по времени: $O(V + E)$ при хранении графа списком смежности, потому что каждая вершина красится один раз, а суммарно по всем запускам DFS просматриваются все записи списков смежности. Цикл по всем вершинам добавляет только $O(V)$ проверок цвета.

2.5 Поиск цикла в графе

Описанный ниже способ с серыми вершинами используется для поиска цикла в ориентированном графе. Чтобы найти цикл, нужно раскрасить граф в три цвета:

- **Белый** — вершина не посещена
- **Серый** — DFS вошел в вершину, но не обработал всех соседей
- **Черный** — все соседи вершины посещены и помечены черным

Обратное ребро в серую вершину означает, что в ориентированном графе есть цикл

Для неориентированного графа нужно учитывать, что ребро из вершины в ее родителя в дереве DFS не образует цикл. Поэтому при просмотре соседа to вершины v цикл найден только если to уже посещен и $to \neq parent[v]$.

Если нужно проверить весь граф, а не только одну достижимую часть, DFS запускают в цикле из всех еще не посещенных вершин.

Алгоритм

- В каждый момент времени серым цветом будут помечены вершины, лежащие на пути **DFS** от стартовой вершины до текущей
- Если из текущей вершины u есть ребро в серую вершину v , то в графе есть цикл, так как существует путь от v до u (v лежит на пути от стартовой вершины до u) и путь от u до v , проходящий по одному ребру

Временная сложность: $O(V + E)$ при хранении графа списком смежности, поскольку каждая вершина и каждая запись в списках смежности обрабатываются не более одного раза

Память: $O(V)$ под массив меток и стек рекурсии или явный стек, плюс хранение графа.

Восстановление цикла

Допустим, для u нашелся серый сосед v , тогда запоминаем номер v и выходим из рекурсивной функции, запоминая номера вершин, пока не дойдем до вершины, в которой нашелся цикл

2.6 Проверка графа на двудольность

Примечание. Граф — двудольный тогда и только тогда, когда все циклы в графе имеют четную длину

Алгоритм

- Выбираем произвольную вершину и красим ее в цвет $color$
- Всех соседей этой вершины красим в цвет $3 - color$
- Соседей соседей — в цвет $color$ и т.д.
- Если в какой-то момент сосед вершины уже покрашен в тот же цвет, что и вершина, то алгоритм завершает работу, так как граф не двудольный, потому что есть циклы нечетной длины

Если граф не является связным, то нужно запустить **DFS** из каждой вершины каждой компоненты связности (как в разделе про поиск компонент связности)

Временная сложность: $O(V + E)$ при хранении графа списком смежности, потому что каждая вершина получает цвет один раз, а каждое ребро проверяется при просмотре списков смежности.

Память: $O(V)$ под массив цветов и стек рекурсии или явный стек, плюс хранение графа.

2.7 Диаметр и центр дерева

Диаметр дерева — максимальная длина (в рёбрах) кратчайшего пути в дереве между любыми двумя вершинами

Центр дерева — одна вершина или две соседние вершины, минимизирующие максимальное расстояние до всех остальных вершин

Примечание. Центр можно понимать так: это вершина дерева, такая что при подвешивании дерева за нее глубина дерева минимальна

Алгоритм поиска диаметра дерева

Для поиска диаметра требуется выполнить два запуска DFS. Этот алгоритм работает именно для дерева.

- Берем любую вершину дерева (пусть это a) и ищем самую удаленную от нее вершину b с помощью **DFS**
- Из вершины b запускаем **DFS** и ищем самую удаленную от нее вершину (пусть это c)
- Путь из b в c — диаметр дерева

Центр дерева находится как середина найденного диаметрального пути: если длина пути в ребрах четная, центр один; если нечетная, центров две соседние вершины.

Временная сложность: $O(V + E) = O(V)$, потому что в дереве $E = V - 1$, а алгоритм выполняет два DFS и затем один раз просматривает найденный путь.

Память: $O(V)$ под массив посещенности, массив предков для восстановления пути и стек рекурсии или явный стек, плюс хранение дерева.

3 Задача построения дерева кратчайших расстояний

3.1 Обход в ширину

Наивный алгоритм

Создаем массив расстояний, заполняем его бесконечностями. Для начальной вершины установим значение 0

Делаем шаги по расстояниям, пока есть вершины текущего слоя. На каждом шаге перебираем все вершины, выбираем те, расстояние до которых равно номеру текущего слоя, и помечаем их еще не посещенных соседей числом на 1 большим, чем номер текущего слоя

- На нулевом шаге выбираем начальную вершину и помечаем ее соседей 1
- Затем выбираем вершины, находящиеся на расстоянии 1, и их непомяченных соседей помечаем 2 и т.д.

Сложность по времени — $O(V^2 + E)$, так как мы сделаем не более V шагов, на каждом шаге переберем V вершин, а также суммарно просмотрим все рёбра

Сложность по памяти — $O(V)$ заводится под массив расстояний

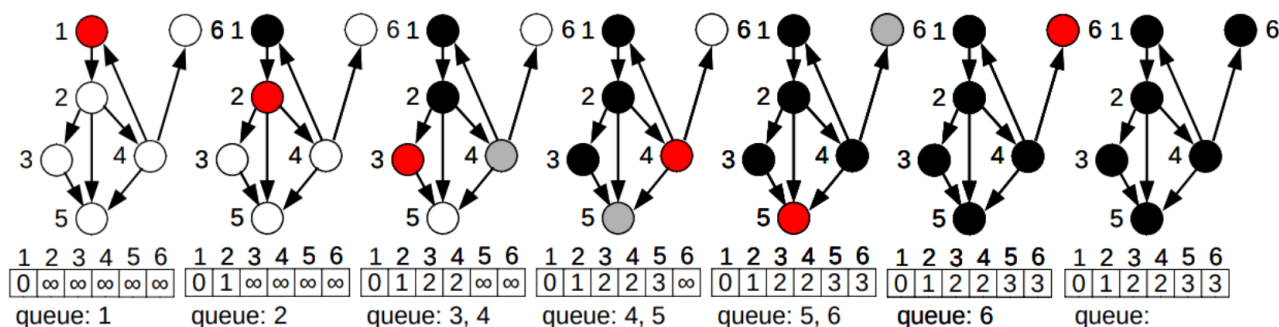
Реализация через очередь

1. Инициализировать для всех u : $\text{dist}[u] = \infty$, $\text{visited}[u] = \text{false}$
2. $\text{dist}[\text{start}] = 0$, $\text{visited}[\text{start}] = \text{true}$, положить start в очередь q
3. Пока q не пуста:
 - $u = q.\text{pop}()$
 - Для каждого соседа v из $\text{adj}[u]$:
 - Если $\neg \text{visited}[v]$:
 - * $\text{visited}[v] = \text{true}$
 - * $\text{dist}[v] = \text{dist}[u] + 1$
 - * $q.\text{push}(v)$
 - * Если v — искомая вершина, выйти из всех циклов (если ищем путь до конкретной вершины)

Сложности:

- Время: $O(V + E)$
- Память: $O(V)$ под очередь, массивы `dist` и `visited`, плюс хранение графа

Белый — еще не посещенные вершины, *черный* — уже посещенный, *красный* — обрабатывающиеся в данный момент (в начале очереди), *серый* — вершины, находящиеся в очереди



3.2 Алгоритм Дейкстры

Важно. Веса рёбер должны быть неотрицательными, иначе алгоритм не будет работать корректно

Случай для плотного графа

Создадим такие массивы

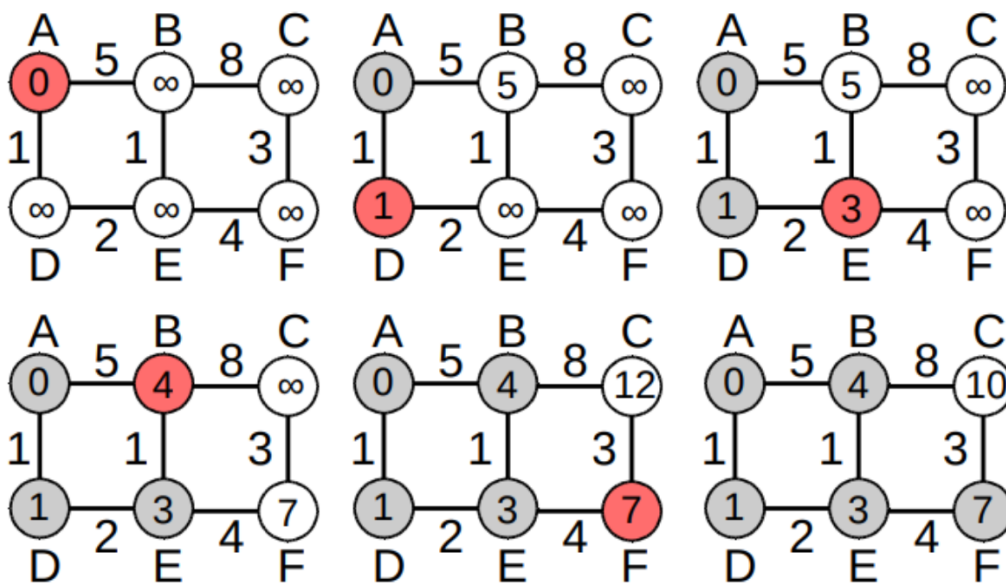
- `dist` — размером $V + 1$. В нем для каждой вершины хранится текущая длина кратчайшего пути от начальной вершины s . (`dist[s] = 0`, а все остальное — ∞)
- `visited` — размером $V + 1$. Хранит информацию о том, обработана вершина или нет

Алгоритм состоит не более чем из V шагов. На каждом шаге делаем следующее

Для $i = 1 \dots V$:

1. Выбрать необработанную вершину v с минимальным `dist[v]`
2. Если такой вершины нет или `dist[v]` равно ∞ , то все достижимые вершины уже обработаны
3. Пометить v как обработанную и выполнить релаксацию всех рёбер $(v \rightarrow u)$:

$$\text{dist}[u] = \min(\text{dist}[u], \text{dist}[v] + w(v, u)).$$



Красный — вершины, для которых будет произведена релаксация, *серый* — обработанные вершины

Сложность со списком смежности — $O(V^2 + E)$, так как мы на каждом из не более чем V шагов ищем минимум из V чисел для вершин и суммарно просматриваем все рёбра

Сложность с матрицей смежности — $O(V^2)$

Случай для разреженного графа

Напоминание. Разреженный граф - граф, у которого количество рёбер значительно меньше, чем V^2

В разреженном графе минимум среди необработанных вершин удобно искать не линейным проходом, а с помощью структуры данных: `ordered set` или очереди с приоритетами.

1. Инициализируем расстояния:

- `dist[s] = 0`

- Для всех остальных вершин `dist[v]` равно ∞
- В структуру данных кладем пару $(0, s)$

2. Пока структура данных не пуста:

- Достаем вершину v с минимальным текущим расстоянием
- Если достали устаревшее значение расстояния, пропускаем его
- Для каждого ребра $(v \rightarrow u)$ веса w выполняем релаксацию:
 - Если `dist[u] > dist[v] + w`, то обновляем `dist[u]`
 - В `ordered set` удаляем старую пару для u и добавляем новую
 - В `priority queue` просто добавляем новую пару; старые пары будут пропущены, когда окажутся наверху

Процесс повторяется, пока не будут определены кратчайшие пути до всех достижимых вершин.

Сложность — $O((V + E) \log V)$ при использовании `ordered set` или кучи с ленивым удалением устаревших значений. Извлечение минимума и обновление структуры данных стоят $O(\log V)$, а каждое ребро может привести не более чем к одной успешной релаксации.

В итоге массив расстояний содержит кратчайшие пути от начальной вершины до всех остальных достижимых вершин.

3.3 Алгоритм Форда-Беллмана

Создадим такой массив

- `dist` — размером $V + 1$. Массив кратчайших расстояний. (`dist[s] = 0`, а все остальное — ∞)

Алгоритм состоит из $V - 1$ фаз. Пусть есть ребро (u, v) и его вес w . На каждой фазе перебираем все рёбра и проводим релаксацию по этому ребру, то есть

```
if (dist[u] != inf && dist[v] > dist[u] + w){
    dist[v] = dist[u] + w
}
```

Проверка отрицательного цикла

После $V - 1$ фаз запускаем еще одну фазу релаксаций. Если на ней расстояние до какой-то вершины снова уменьшилось, то существует отрицательный цикл, достижимый из стартовой вершины. В этом случае кратчайшие расстояния до вершин, на которые влияет этот цикл, не определены.

Применение алгоритма

Алгоритм Форда-Беллмана ищет кратчайшие пути от одной вершины до всех остальных, работает при отрицательных рёбрах и умеет обнаруживать отрицательный цикл, достижимый из стартовой вершины.

Временная сложность: $O(V \times E)$

Сложность по памяти: $O(V + E)$

- $O(V)$ под массив `dist`
- $O(E)$ под список рёбер

3.4 Алгоритм Флойда–Уоршалла

Работаем с матрицей размера $V \times V$, где

$$\text{dist}[i][j] = \begin{cases} w(i, j), & \text{если } (i \rightarrow j) \text{ есть ребро,} \\ 0, & i = j, \\ \infty, & \text{иначе.} \end{cases}$$

Основной тройной цикл:

```
for k in 1..V:
    for i in 1..V:
        for j in 1..V:
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Обнаружение отрицательного цикла:

Если после выполнения алгоритма $\text{dist}[i][i] < 0$ для некоторого i , в графе есть отрицательный цикл.

Применение

- Поиск кратчайших путей «каждый→каждый»

Сложности:

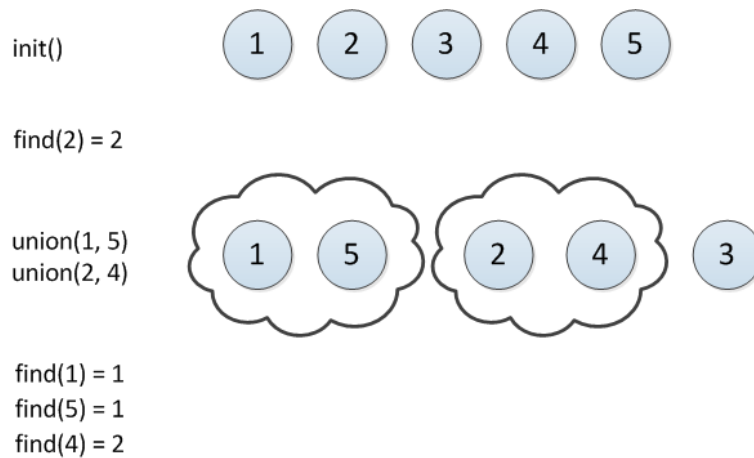
- Время: $O(V^3)$
- Память: $O(V^2)$

4 Задача union — find

4.1 Задача union — find

Пусть у нас есть N элементов, занумерованных от 1 до N . Изначально каждый элемент находится в своем отдельном множестве (также занумерованных от 1 до N). Нам необходимо поддерживать такие операции:

- `find(x)` — найти номер множества, в котором лежит x
- `union(x, y)` — объединить множества, содержащие x и y
- Можно добавить, но необязательно:
 - `make(x)` — создает новое множество, содержащее x



4.2 Наивная реализация

1. Создаем массив с индексами от 1 до N
 - Индекс означает номер элемента
 - Значение по индексу — номер множества, к которому относится элемент
2. Операция `find(x)`
 - Нужно вернуть содержимое ячейки с номером x
 - Сложность — $O(1)$
3. Операция `union(x, y)`
 - Узнаем номера множеств, содержащих эти элементы (пусть это a и b соответственно)
 - Проходим по всему массиву и заменяем значения b на a
 - Сложность одной операции — $O(N)$
 - Сложность объединения всех множеств — $O(N^2)$

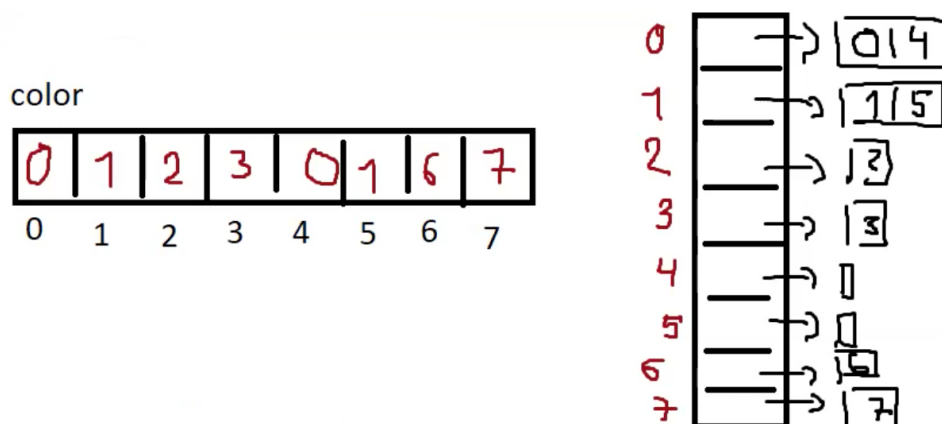
4.3 Реализация с использованием линейных списков

Идея этой реализации заключается в том, что мы заводим второй массив, на индексах которого стоят номера множеств, а по индексу хранится список элементов соответствующего множества

Тогда при вызове `union(x, y)` мы делаем так: `max(x, y) += min(x, y)`

Сложность

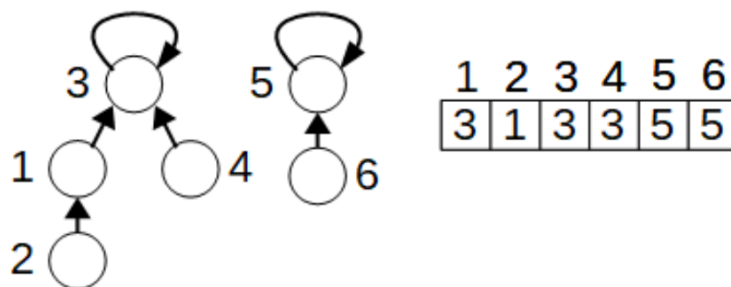
Так как мы храним второй массив, то мы *тратим больше памяти*, однако алгоритм будет работать *быстрее*, за $O(N \log N)$, так как делаем $O(\log N)$ шагов на каждом шаге $O(N)$ для объединения множеств



4.4 Система непересекающихся множеств

Идея заключается в хранении каждого множества в виде дерева

Мы имеем единственный массив предков **p**. Индексы в нем — номера элементов, а по индексу хранится номер предка



Петля у вершины означает, что вершина является корнем дерева, а в **p** на позицию с номером корня записывается значение самого корня

Вместо этого, в **p** на позицию корня можно поставить -1 , суть не изменится

Тогда операции **find(x)** и **union(x, y)** можно реализовать так:

```
int find(x){
    while (p[v] != -1){
        v = p[v]
    }
    return v
}

void union(x, y){
    p[find(x)] = find(y)
}
```

Сложность — $O(N^2)$

4.4.1 Сжатие путей

Идея заключается в том, что после поиска корня множества с помощью **find(x)** мы записываем этот корень в **parent** для вершины x и всех вершин, пройденных по пути

Теперь в массиве предков для каждой вершины там может храниться не непосредственный предок, а предок предка, предок предка предка, и т.д

Сложность — $O(\log N)$

4.4.2 Объединение деревьев

Идея: нужно подвешивать дерево с большей глубиной к дереву с меньшей глубиной

Для каждого дерева храним его глубину в массиве

Сложность — $O(\log N)$

4.5 Алгоритм Краскала

- Отсортируем все рёбра по возрастанию
- Каждая вершина находится в своем отдельном множестве
- В остовное дерево берем те рёбра, которые соединяют разные множества вершин
- При добавлении ребра происходит объединение множеств

Алгоритм используется для построения минимального остовного дерева в графе

Реализовать представленный алгоритм проще всего с помощью СНМ. Необходимо отсортировать рёбра по неубыванию по их весам. Далее мы каждую вершину можем поместить в свое собственное дерево, то есть, создаем некоторое множество подграфов. Дальше итерируемся по всем рёбрам в отсортированном порядке и смотрим, принадлежат ли инцидентные вершины текущего ребра разным подграфам с помощью функции `find()` или нет, если оба конца лежат в разных компонентах, то объединяем два разных подграфа в один с помощью функции `union()`.

Итоговая сложность составит $O(E \log V + V + E) = O(E \log V)$, где $O(E \log V)$ - сортировка, $O(V)$ — создание отдельных множеств, $O(1)$ — `find()` и `union()`

5 Дерево отрезков

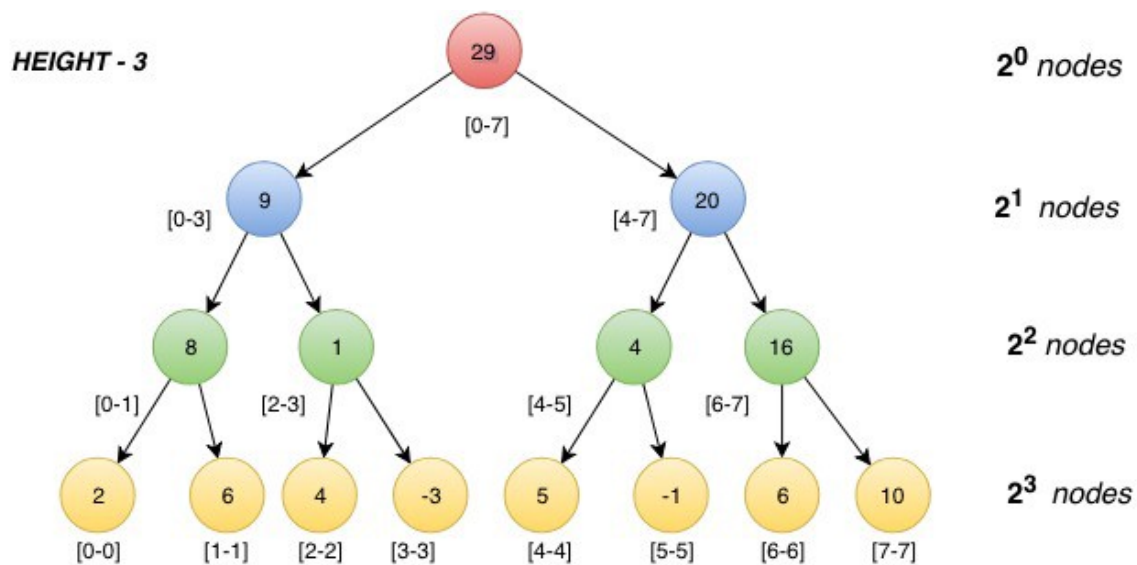
5.1 Дерево отрезков

Дан массив a из n целых чисел, и требуется отвечать на запросы двух типов:

1. Изменить значение в ячейке (т. е. реагировать на присвоение $a[k] = x$)
2. Вывести сумму элементов a_i на отрезке с l по r

Построим вспомогательное дерево

- Посчитаем сумму всего массива и сохраним ее
- Разделим его пополам, посчитаем сумму на половинах и тоже сохраним
- Каждую половину разделим пополам ещё раз, и т.д., пока не придём к отрезкам длины 1



Корень этого дерева соответствует отрезку $[0, n)$, а каждая вершина (не считая листьев) имеет ровно двух сыновей, которые тоже соответствуют каким-то отрезкам

5.2 Операции на отрезке

Здесь представлены операции на отрезке, реализованные с помощью указателей. Эта реализация не самая эффективная, но самая простая, решающая большинство задач. Подробнее о других реализациях [тут](#) и [тут](#)

5.2.1 Построение

Строить дерево отрезков можно рекурсивным конструктором, который создает детей, пока не доходит до листьев

Если изначально массив не нулевой, то можно сразу при построении вычислять суммы

Сложность — $O(n)$

```
Segtree(int lb, int rb) : lb(lb), rb(rb) {
    if (lb + 1 == rb)
        s = a[lb];
    else {
        int t = (lb + rb) / 2;
        l = new Segtree(lb, t);
```

```

        r = new Segtree(t, rb);
        s = l->s + r->s;
    }
}

```

5.2.2 Изменение

Для запроса прибавления будем рекурсивно спускаться вниз, пока не дойдем до листа, соответствующего элементу k , и на всех промежуточных вершинах прибавим x :

```

void add(int k, int x) {
    s += x;
    if (l){
        if (k < l->rb)
            l->add(k, x);
        else
            r->add(k, x);
    }
}

```

Сложность — $O(\log n)$

5.2.3 Сумма

Нужно делать разбор случаев, как отрезок запроса пересекается с отрезком вершины:

1. Если лежит полностью в отрезке запроса, вывести сумму
2. Если не пересекается с отрезком запроса, вывести ноль
3. Иначе: рекурсивно запускаемся от детей

```

int sum(int lq, int rq) {
    if (lb >= lq && rb <= rq)
        return s;
    if (max(lb, lq) >= min(rb, rq))
        return 0;
    return l->sum(lq, rq) + r->sum(lq, rq);
}

```

Сложность — $O(\log n)$. На каждом уровне дерева отрезков наша рекурсивная функция может посетить максимум четыре отрезка; тогда, учитывая оценку $O(\log n)$ для высоты дерева, мы получаем асимптотику времени работы алгоритма

5.3 Применение дерева отрезков

Дерево отрезков может применяться в таких задачах, как

- поиск суммы на подотрезке
- поиск минимума/максимума на отрезке
- массовые изменения массивов (например, прибавление значения ко всем элементам отрезка)

6 Дерево поиска

Бинарное дерево поиска — дерево, для которого выполняются следующие свойства:

- У каждой вершины не более двух детей
- Все вершины обладают *ключами*, на которых определена операция сравнения (например, целые числа или строки)
- У всех вершин *левого* поддерева вершины v ключи *не больше*, чем ключ v
- У всех вершин *правого* поддерева вершины v ключи *больше*, чем ключ v
- Оба поддерева — левое и правое — являются бинарными деревьями поиска

В *небинарных* деревьях количество детей может быть больше двух, и при этом в «более левых» поддеревах ключи должны быть меньше, чем «более правых»

Для работы с деревьями поиска нужно создать структуру

```
struct Node:
    T key           // key of the node
    Node left       // pointer to the left child
    Node right      // pointer to the right child
    Node parent     // pointer to the parent
```

6.1 Поиск элемента

Нужна функция, принимающая корень дерева и искомый ключ

- Для каждого узла сравниваем значение его ключа с искомым ключом
- Если ключи одинаковы, то функция возвращает текущий узел
- В противном случае функция вызывается рекурсивно для левого или правого поддерева

```
Node search(x : Node, k : T):
    if x == null or k == x.key
        return x
    if k < x.key
        return search(x.left, k)
    else
        return search(x.right, k)
```

Сложность в худшем случае — $O(h)$ (h — высота дерева), так как узлы, которые посещает функция образуют нисходящее дерево. Такое возможно, когда дерево является «бамбуком»

Сложность в сбалансированном дереве — $O(\log N)$, потому что высота такого дерева равна $O(\log N)$

6.2 Вставка элемента

Почти то же самое, что поиск элемента, но теперь, когда у текущей вершины нет нужного ребенка, нужно подвесить на нее вставляемый элемент

```
Node insert(x : Node, z : T):           // x - root of the subtree, z - key to be inserted
    if x == null
        return Node(z)                  // attach a Node with key = z
    else if z < x.key
        x.left = insert(x.left, z)
    else if z > x.key
        x.right = insert(x.right, z)
    return x
```

Сложность в худшем случае — $O(h)$ (h — высота дерева): как и при поиске, алгоритм проходит по одному пути от корня до места вставки. В несбалансированном дереве высота может быть равна $O(N)$.

Сложность в сбалансированном дереве — $O(\log N)$, потому что высота такого дерева равна $O(\log N)$.

6.3 Удаление элемента

Рассмотрим три случая при рекурсивной реализации

1. Удаляемый элемент находится в *левом* поддереве текущего поддерева
 - тогда нужно рекурсивно удалить элемент из нужного поддерева
2. Удаляемый элемент находится в *правом* поддереве
 - тогда нужно рекурсивно удалить элемент из нужного поддерева
3. Удаляемый элемент находится в *корне*; есть два случая:
 - у него два дочерних узла
 - нужно заменить его минимальным элементом из правого поддерева и рекурсивно удалить этот минимальный элемент из правого поддерева
 - у него один дочерний узел
 - нужно заменить удаляемый элемент потомком

```
Node delete(root : Node, z : T): // root of subtree, key to delete
if root == null
    return root
if z < root.key
    root.left = delete(root.left, z)
else if z > root.key
    root.right = delete(root.right, z)
else if root.left != null and root.right != null
    root.key = minimum(root.right).key
    root.right = delete(root.right, root.key)
else
    if root.left != null
        root = root.left
    else if root.right != null
        root = root.right
    else
        root = null
return root
```

Сложность в худшем случае — $O(h)$ (h — высота дерева). Сначала алгоритм спускается к удаляемой вершине, а если у нее два ребенка, дополнительно ищет минимум в правом поддереве; это всё равно движение по пути вниз и не меняет оценку. В несбалансированном дереве высота может быть равна $O(N)$.

Сложность в сбалансированном дереве — $O(\log N)$.

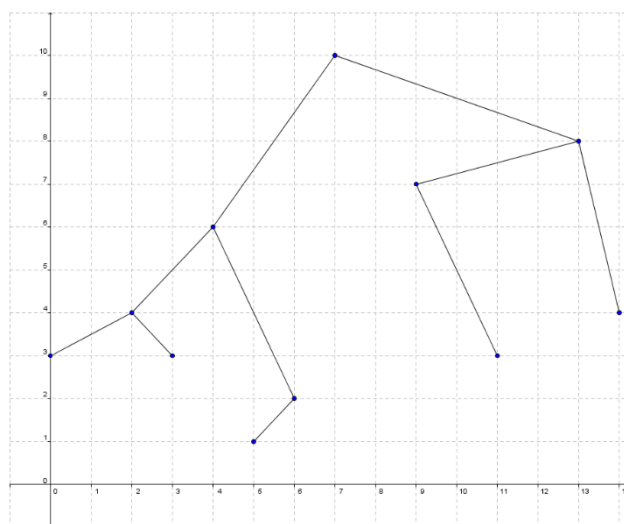
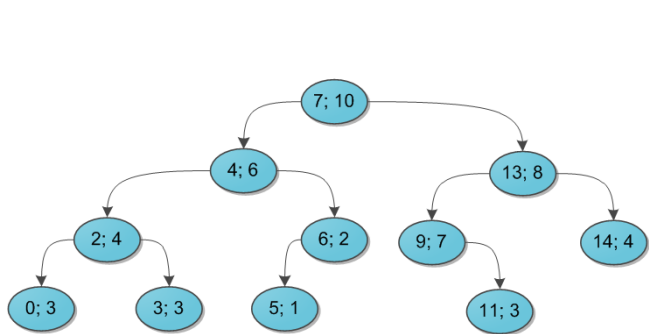
7 Декартово дерево

7.1 По явному ключу

Имеется бинарное дерево. Каждый его узел (обозначим как ключ x) дополняем случайным числом y . По ключам x образуется бинарное дерево поиска, а структура, полученная по y , обладает свойством двоичной кучи максимумов

Еще способ:

- Нанесём на плоскость набор из n точек. Их x назовем *ключом*, а y *приоритетом*
- Выберем самую верхнюю точку (с наибольшим y , а если таких несколько — любую) и назовём её *корнем*
- От всех вершин, лежащих слева (с меньшим x) от корня, рекурсивно запустим этот же процесс. Если слева была хоть одна вершина, то присоединим корень левой части в качестве левого сына текущего корня
- Аналогично, запустимся от правой части и добавим корню правого сына



Одно и то же дерево в обычном представлении и в виде декартового дерева

При добавлении второго ключа y представление дерева становится однозначным

Например, правым сыном корня будет самая верхняя вершина, находящаяся справа от корня, а левым — самая верхняя вершина слева от корня

Сложность — $O(\log N)$, так как координаты y выбираются случайно, то дерево получается сбалансированным и его высота равна $O(\log N)$

7.2 По неявному ключу

Работа с ДД по неявному пригодится в случае, если есть такие операции:

1. выводить сумму на произвольном отрезке
2. «переворачивать» произвольный отрезок, то есть переставлять элементы с l по r в обратном порядке, не меняя остальные

Так как во второй операции невозможно быстро поддерживать ключи актуальными, то делаем следующее:

- Убираем ключи

- Вместе с каждой вершиной храним размер ее поддеревя
- Ключ (позицию элемента) восстанавливаем как число элементов слева от него
- Размер деревьев поддерживаем с помощью вспомогательной функции, вызываемой после каждого изменения дерева:

```
int size(Node *v) { return v ? v->size : 0; }

void upd(Node *v) { v->size = 1 + size(v->l) + size(v->r); }
```

Сложность — $O(\log N)$

7.3 Операции

Создадим структуру `Node`, в которой будем хранить ключ, приоритет и указатели на левого и правого сына

Сначала описывается реализация по явному ключу, после — по неявному

7.3.1 Merge

Принимает два дерева (два корня, L и R), про которые известно, что в левом все вершины имеют меньший ключ, чем все в правом. Их нужно объединить в одно дерево так, чтобы по ключам это было всё ещё дерево, а по приоритетам — куча

Сначала выберем, какая вершина будет корнем. Возьмем ту, у которой приоритет больше

Пусть это будет левый корень. Тогда левый сын корня итогового дерева должен быть левым сыном L . С правым сыном сложнее: возможно, его нужно смержить с R . Поэтому рекурсивно сделаем `merge(l->r, r)` и запишем результат в качестве правого сына

```
Node* merge (Node *l, Node *r) {
    if (!l) return r;
    if (!r) return l;
    if (l->prior > r->prior) {
        l->r = merge(l->r, r);
        return l;
    }
    else {
        r->l = merge(l, r->l);
        return r;
    }
}
```

По неявному ключу операция не изменится, так как нигде не использует ключи

7.3.2 Split

Принимает дерево и ключ x , по которому его нужно разделить на два: L должно иметь все ключи не больше x , а R должно иметь все ключи больше x

В этой функции мы сначала решим, в каком из деревьев должен быть корень, а потом рекурсивно разделим его правую или левую половину и присоединим, куда надо:

```
typedef pair<Node*, Node*> Pair;

Pair split (Node *p, int x) {
    if (!p) return {0, 0};
    if (p->key <= x) {
        Pair q = split(p->r, x);
        p->r = q.first;
        return {p, q.second};
    }
}
```

```

    }
    else {
        Pair q = split(p->l, x);
        p->l = q.second;
        return {q.first, p};
    }
}

```

По неявному ключу операция немного изменяется. Теперь нужно использовать позицию корня вместо ключа. `Split` теперь выполняет такую команду, как «вырежи первые k элементов»

```

pair<Node*, Node*> split(Node *p, int k) {
    if (!p) return {0, 0};
    if (size(p->l) + 1 <= k) {
        auto [l, r] = split(p->r, k - size(p->l) - 1);
        p->r = l;
        upd(p);
        return {p, r};
    }
    else {
        auto [l, r] = split(p->l, k);
        p->l = r;
        upd(p);
        return {l, p};
    }
}

```

7.4 Применение декартового дерева

Операции `Merge` и `Split` позволяют реализовать следующие операции

7.4.1 Вставка

Например, чтобы добавить число x в дерево, мы можем разрезать его по x через `split`, создать новую вершину с одним числом x , и склеить через `merge` три получившихся дерева

7.4.2 Сумма на отрезке

- При `merge` и `split` надо будет поддерживать эту сумму актуальной
- Можно написать вспомогательную функцию `upd`, которую будем вызывать при обновлении детей вершины
- В `merge` и `split` теперь можно просто вызывать `upd` перед тем, как вернуть вершину
- При запросе суммы нужно вырезать нужный отрезок и запросить эту сумму

7.4.3 Переворот

Добавим флаг `rev` в каждую вершину, означающий, что подотрезок был перевернут

Когда мы встретим такую вершину, мы поменяем местами ссылки на её детей, а им самим передадим эту метку

Тогда функция `reverse` будет такой: вырезаем нужный отрезок, меняем флаг